TECHNICAL DESIGN DOCUMENT

Money Movement Application

Django REST Framework + React + PostgreSQL — architecture, data model, API surface, and frontend build plan for a 2-person, 6-hour hackathon build.

EVENT	PSTU IT Carnival 2025 — Money Movement Application Challenge
TEAM SIZE	2 developers
DURATION	9:00 AM – 3:00 PM (6 hours)
STACK	Django + Django REST Framework, React, PostgreSQL

CONTENTS

1. How to read the two source docs

The requirements doc is the organizers' own reference spec — likely what gets graded against. The challenge brief is intentionally open ("we intentionally leave the detailed product requirements open"), but the requirements doc has already interpreted it into an exact FR/NFR checklist. Build to the requirements doc directly.

The one line that matters most: the sum of every account balance must always equal 100,000 × the number of registered users. That's a literal query someone can run against the database at any time. It means no code path may ever create or destroy money — no manual balance edits, no "adjustment" endpoints, ever, even to fix a mistake.

2. Architecture decisions (and why, given 6 hours)

Decision	Choice	Reason
Auth	DRF Token Authentication (not JWT)	One line in settings, no refresh-token flow to build. Login/logout only needs a token that exists or doesn't.
User & balance	A single custom User model, with balance as a field on it	A separate Account model 1:1 with User is "more correct" but is one more join, one more serializer, one more thing to get wrong under time pressure.
Money movement	One Transaction model, used for both direct sends and approved requests	A send and an approved request must execute through the same code path — one model keeps that guarantee structural instead of hoped-for.
Concurrency	Row-level locking on both accounts inside a single database transaction, locks acquired in a consistent sorted order	Prevents double-spending under simultaneous requests, and prevents two opposite-direction transfers between the same pair of users from deadlocking each other.
Idempotency	A unique client-generated key stored per transaction attempt	A duplicate double-tap or network retry returns the original result instead of moving money twice.
Pagination	Standard page-number pagination on the history endpoint	Transaction history is never loaded in full in one request.
Django apps	Two apps: `accounts` and `ledger`	Enough separation to reason about; not enough to lose time on cross-app plumbing in a 6-hour window.

Do not build: JWT refresh flow, Celery/async workers, WebSockets or live push, Redis, microservices, real bank integration, or an ML fraud model — the requirements doc explicitly says a trained model isn't feasible in the time available and asks for a rule-based engine instead.

3. Django apps

3.1 `accounts`

Owns the User model — a registered person's login credentials and their balance. Registration, login, logout, and "who am I" all live here. Balance is a field on this model, so crediting the initial 100,000 on signup is a single record creation with no follow-up step.

3.2 `ledger`

Owns everything about money movement and transaction history — the Transaction model, the transfer logic, requests, approvals, declines, cancellations, and the history/detail views. Every read and write related to a transaction lives in this one app, which is what keeps “sends and approved requests share one execution path” enforceable rather than just documented.

4. Data model

4.1 User (in `accounts`)

Field	Type	Notes
username	string	unique, indexed — required for lookup by exact username
password	hashed string	never stored or returned in plaintext
balance	decimal	defaults to 100,000.00 at creation time

Balance is never exposed in any API response except the requesting user’s own profile — a user must never be able to see another user’s balance.

4.2 Transaction (in `ledger`)

Field	Type	Notes
id	UUID	primary key
type	choice	`send` or `request`
status	choice	`pending`, `completed`, `declined`, `cancelled`, or `failed`
initiator	FK → User	whoever performed the API call — the sender for a send, the requester for a request
counterparty	FK → User	the other party — for a request, this is the person who will pay if they approve it
amount	decimal	must be strictly greater than zero
note	text	optional
idempotency_key	UUID	unique — the field the duplicate-tap protection is built on
risk_flagged	boolean	stretch goal only, defaults to false
risk_reason	text	stretch goal only, optional
created_at	datetime	set automatically
resolved_at	datetime	set when the transaction leaves the pending state

Indexed on initiator, on counterparty, and on status, so that “my history,” “my pending requests,” and filtering by status all stay fast as the table grows.

`initiator` / `counterparty` are used instead of `sender` / `receiver` deliberately: for a **send**, the initiator pays. For a **request**, the initiator is *requesting* money — the counterparty is the one who pays, if and when they approve. Naming it this way in the model avoids a whole class of “who pays whom” bugs at approval time.

No hard deletes anywhere in this app, and no endpoint ever removes a transaction — every record, once created, stays in the table permanently. This is what makes the transaction history a genuine audit trail rather than something that can be tampered with after the fact.

5. How the transfer and request logic works

This is the highest-value part of the whole build — it's what makes the balance invariant, the no-negative-balance rule, and the no-double-spend rule actually true rather than just assumed.

Moving money between two accounts, in general: 1. Lock both accounts involved, inside a single database transaction, always in the same consistent order (e.g. sorted by account ID) regardless of who is paying whom. Locking in a consistent order is what prevents two simultaneous transfers between the same pair of users from deadlocking each other. 2. Re-check the payer's balance *after* the lock is acquired, not before — checking beforehand would leave a window where a concurrent transfer could slip through and cause a negative balance. 3. If the balance is insufficient, stop here and report failure. Nothing is written. 4. Otherwise, debit the payer and credit the payee as part of the same database transaction, so a crash or error between the two updates is impossible — either both happen, or neither does.

This single operation is the *only* thing in the entire system allowed to change a balance. A direct send calls it once, immediately. An approved request calls it once, at the moment of approval. Nothing else touches balances.

Direct send: Look up the transaction by the client-supplied idempotency key first. If a matching one already exists, return it as-is — don't move money again. If it's new, attempt the transfer described above; mark the result completed or failed accordingly.

Creating a request: Create a pending transaction record. No money moves at this point — it's purely a record of who is asking whom for how much.

Approving a request: Only the counterparty (the person being asked to pay) may approve it, and only while it's still pending. On approval, run the same transfer operation, with the counterparty paying the initiator. Mark the result completed or failed.

Declining a request: Only the counterparty may decline, and only while pending. No money moves; the record is marked declined and stays visible in history.

Cancelling a request: Only the initiator (the person who asked) may cancel their own outgoing request, and only while it's still pending. No money moves; the record is marked cancelled.

Duplicate protection: Every send and every request-creation call carries a client-generated idempotency key. If the same key arrives twice — a double-tap, a retried network call — the second call returns the original result instead of executing anything a second time. This is enforced at the database level via a uniqueness constraint on that key, not just checked in application code, so it holds even under simultaneous requests.

6. API endpoints

Base path `/api/`. All responses are JSON. Every account and money-movement endpoint requires an auth token except register and login.

6.1 `accounts`

Method	Endpoint	Purpose
POST	`/api/auth/register/`	Create a user; balance is credited automatically at 100,000
POST	`/api/auth/login/`	Authenticate, receive a token
POST	`/api/auth/logout/`	Invalidate the current token
GET	`/api/accounts/me/`	Current user's username and balance — never another user's

6.2 `ledger`

Method	Endpoint	Purpose
POST	`/api/transactions/send/`	Send money directly to another user by username
POST	`/api/transactions/request/`	Request money from another user by

		username — creates a pending record
POST	`/api/transactions/{id}/approve/`	Approve a pending request addressed to you — executes the transfer
POST	`/api/transactions/{id}/decline/`	Decline a pending request addressed to you — no money moves
POST	`/api/transactions/{id}/cancel/`	Cancel your own pending outgoing request
GET	`/api/transactions/`	Paginated transaction history, filterable by status and type
GET	`/api/transactions/{id}/`	Detail of a single transaction — only visible to its two parties
GET	`/api/transactions/pending/incoming/`	Requests currently awaiting your action, as the payer
GET	`/api/transactions/pending/outgoing/`	Your own requests currently awaiting the other party's action

Validation rejected with a clear error message, not a stack trace, for: - amount that is zero, negative, or not a valid number - sending or requesting from yourself - a username that doesn't exist - insufficient balance at the moment of a send or an approval (a distinct case from the above — it's a valid request that fails a business rule, not malformed input)

Access control enforced on every endpoint above: - a user can only ever see their own balance and their own transaction history - a user can only approve or decline a request addressed to them - a user can only cancel a request they themselves created

7. Frontend (React) — pages

Page	Route	What it contains
Login	`/login`	Username and password fields, submit action, link to Register, inline auth-error display
Register	`/register`	Username and password fields, submit action; on success, clearly surfaces the initial 100,000 balance credit
Dashboard	`/`	Current balance shown prominently, primary Send and Request actions, a short list of the most recent transactions, a badge showing the count of pending incoming requests
Send Money	`/send` (modal)	Recipient-username field, amount field, optional note field, submit action, inline errors for self-send, unknown username, insufficient balance, and invalid amount
Request Money	`/request` (modal)	Payer-username field, amount field, optional note field, submit action, same inline validation surface as Send
Transaction History	`/history`	Full paginated list, filter controls for type (sent / received / request) and status, each row links to its detail view
Transaction Detail	`/transactions/:id` (modal or route)	Counterparty, amount, direction, status, timestamp, and note; if the record is a pending request the current user can

		act on, shows Approve/Decline (if they're the payer) or Cancel (if they're the requester) inline
Pending Requests	`/requests`	Two tabs — Incoming (payer's view, with Approve/Decline per row) and Outgoing (requester's view, with Cancel per row)

For a 6-hour build, launch Send and Request as modals from the Dashboard rather than separate routed pages — it saves routing and layout work, and matches the natural interaction anyway.

Idempotency key on the frontend: generate a fresh key once when the Send or Request form opens, and reuse the same key across retries of that same in-flight submission — a fresh key on every retry would defeat the duplicate-tap protection entirely. Generate a new key only when the form is reopened for a new attempt.

8. User flows

Registration → first look at the app 1. User submits a username and password on Register. 2. The account is created with a balance of 100,000. 3. The frontend stores the auth token, redirects to the Dashboard, and shows the balance with a brief "you've been credited" message.

Sending money (happy path) 1. Dashboard → Send → modal opens, idempotency key generated. 2. User enters recipient username, amount, optional note, and submits. 3. On success: modal closes, balance refreshes, the new transaction appears at the top of recent activity, and a confirmation toast is shown. 4. On a validation or balance error: the error is shown inline in the modal and the form stays open for correction.

Requesting money 1. Dashboard → Request → modal, enter payer's username, amount, and note, submit. 2. The request is created as pending and confirmed with a toast. 3. It appears in the requester's Outgoing tab, and in the payer's Incoming tab the next time they load it.

Responding to a request (payer's side) 1. Payer opens Pending Requests → Incoming tab and sees the request with Approve/Decline actions. 2. Approve: on success, the payer's balance decreases, and the record moves out of the pending lists into history as completed. If the payer's balance has since become insufficient, an inline error is shown and nothing moves. 3. Decline: the record is marked declined, no money moves, and it disappears from both parties' pending lists while remaining visible in full history.

Cancelling your own outgoing request 1. Requester opens Pending Requests → Outgoing tab and cancels a still-pending row. 2. The record is marked cancelled and removed from both pending lists.

Viewing history 1. History page loads a paginated list; filter controls call the same endpoint with different query parameters. 2. Clicking a row opens its detail view; if it's still pending and the current user can act on it, the relevant action buttons appear there too.

9. Build order for 6 hours, 2 people

Split by layer — one person owns backend end-to-end, the other owns frontend end-to-end. Agree on the endpoint list in section 6 in the first 15–20 minutes and don't deviate from it without telling each other.

Together, first 20 minutes: agree on the endpoint contract, scaffold both repos, get Postgres running locally.

Backend, next ~3 hours: 1. `accounts` app — User model, migration, register/login/logout/me. 2. `ledger` app — Transaction model and migration. 3. The core transfer operation and direct-send flow, including locking and idempotency — this is the part worth the most care, since it's what a judge will pressure-test. 4. Request creation, approval, decline, and cancellation. 5. Wiring all endpoints from section 6 to views and serializers.

Backend, following hour: validation edge cases, access-control checks, and a quick manual concurrency check — fire two simultaneous sends from the same account and confirm no double-spend occurs.

Frontend, first ~40 minutes: routing skeleton, auth state handling, Login/Register pages — build against stubbed responses if the backend isn't ready yet.

Frontend, next ~2.5 hours: Dashboard, Send/Request modals, History page, Pending Requests page with tabs, Transaction Detail — swapped onto real endpoints as the backend delivers them.

Together, next hour: integration — wire real backend responses into every screen, fix any contract mismatches that came up.

Together, next 45 minutes: error states, loading states, and small but easy-to-miss details like the balance not refreshing after a send; a basic styling pass.

Remaining ~30 minutes: buffer. If the core is fully working and stable, this is where the rule-based fraud detection stretch goal (section 10) goes — otherwise, skip it. A working core beats an incomplete core plus a half-built stretch goal, and the requirements doc explicitly marks it as time-permitting.

Final minutes: deploy and smoke-test, and prepare to explain the locking and idempotency design out loud — that's very likely what differentiates this submission from other teams tackling the same challenge.

10. Stretch goal: rule-based fraud flagging

Only attempt this if section 9 finishes early. Evaluate synchronously, right after a transaction completes — no background queue needed at this scale. A transaction is flagged, never blocked, based on:

- **Large relative to balance** — the amount exceeds a set percentage (e.g. 50%) of the sender's balance before the transaction.
- **High frequency** — the same sender has made several transfers within a short recent window (e.g. 5+ in 5 minutes).
- **Round-trip pattern** — money sent from A to B is immediately sent back from B to A within a short window.
- **Unusual amount for the user** — the amount deviates significantly from that user's own historical average (e.g. more than 3× it).

A flagged transaction is stored with a reason and surfaced as a small badge in the history view — it is never rejected automatically, only marked for review.

11. What to say when judges ask "how do you know this is correct?"

Three answers, each mapped directly to a requirement: 1. **"No negative balances, ever, under concurrent load"** — point to the row-level locking and consistent lock ordering in the core transfer operation. 2. **"No double-spend on duplicate taps"** — point to the unique idempotency key enforced at the database level. 3. **"Total money in the system never changes"** — run a live query summing every balance and show it equals 100,000 × the number of registered users, exactly the invariant the requirements doc specifies.